UNIVERSIDAD DE CUNDINAMARCA
ESPECIALIZACIÓN EN INTELIGENCIA ARTIFICIAL
PROCESAMIENTO DE LENGUAJE NATURAL
DOCENTE: PAUL ALEXANDER DÍAZ MONTAÑA

INFORME DE PROYECTO: ESTADÍSTICAS DE YOUTUBE – AVANCES IV
RESULTADOS Y EVALUACIÓN FRENTE AL DISEÑO INICIAL

ESTUDIANTE: YOHAN NICOLÁS GÓMEZ CASTAÑEDA
CC: 1233903906

FECHA: 25 DE OCTUBRE DE 2026
REPOSITORIO: https://github.com/Nicolas2322/proyecto-estadisticas-youtube
GOOGLE COLAB: https://colab.research.google.com/github/Nicolas2322/proyecto-estadisticas-youtube/blob/main/notebooks/Avances_IV_proyecto_final.ipynb

Contenido

1. Introducción 3
2. Respuestas a las Preguntas de la Entrega 3
3. Diseño Inicial y Solución Final 4
4. Resultados frente al Diseño Inicial 5
    4.1 Etapa A: Clasificación de Sentimiento 6
    4.2 Etapa B: Desempeño del Video 7
    4.3 Modelo Consumible 9
    4.4 Cumplimiento de lo Propuesto en el Diseño 10
5. Retos del Proyecto y su Solución 10
6. Cambio de la Solución Final respecto al Diseño Inicial 12
7. Lecciones Aprendidas 13
8. Trabajo Colaborativo 14
9. Herramientas y Reproducibilidad 14
10. Conclusiones 15
11. Referencias 16

Proyecto Estadísticas de YouTube – Avances IV: Resultados y Evaluación frente al Diseño Inicial

1. Introducción

El presente informe cierra el proyecto de la asignatura, cuyo propósito fue construir un modelo de procesamiento de lenguaje natural (PLN) que estime el desempeño de un video de YouTube a partir de sus comentarios, con el conjunto de datos *YouTube Statistics* de Kaggle (Patil, 2022): 1.867 videos de 41 categorías y 18.139 comentarios etiquetados como negativos, neutrales o positivos, después de la limpieza. Esta cuarta etapa se enfoca en los resultados y en su evaluación con referencia al diseño inicial planteado en Avances II, en los retos que se presentaron a lo largo de todo el proyecto y en la forma en que se resolvieron, en la magnitud del cambio entre el diseño inicial y la solución final y en las lecciones aprendidas.

La solución final se ejecutó de punta a punta en un único notebook de Google Colab con GPU (Tesla T4), publicado en el repositorio de GitHub del proyecto, y su resultado se puede consumir mediante una interfaz web con enlace público. El clasificador de sentimiento pasó de una máquina de soporte vectorial (SVM) con TF-IDF a un modelo de lenguaje RoBERTa entrenado con tuits y ajustado con las etiquetas del proyecto, lo que elevó el F1-macro de 0,710 a 0,901 (+0,191). El modelo de desempeño pasó de un XGBoost individual a un *stacking* de cuatro modelos con intervalos de predicción conformales; su R^2 sobre el logaritmo de las vistas pasó de 0,788 a 0,791, con una cobertura empírica del intervalo del 90 % de 0,901. El hallazgo central del diseño inicial se mantuvo: el tono de los comentarios explica muy poco del alcance de un video, mientras que la interacción que generan los comentarios, la antigüedad y la categoría concentran la capacidad predictiva.

2. Respuestas a las Preguntas de la Entrega

La Tabla 1 sintetiza la respuesta a cada una de las preguntas planteadas para la semana 10; las secciones siguientes desarrollan cada punto.

Tabla 1

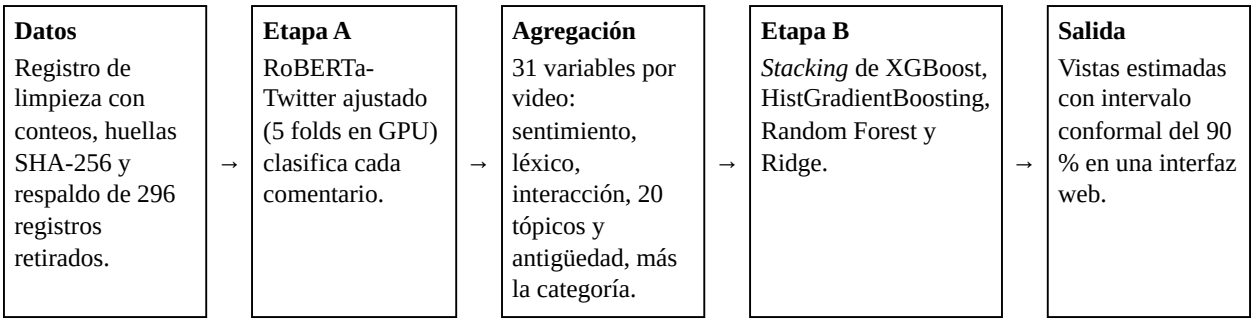
Respuestas a las preguntas de la entrega

Pregunta	Respuesta
Resultados frente al diseño inicial	Etapa A: F1-macro de 0,710 a 0,901 y exactitud de 0,784 a 0,927; la clase negativa, la más difícil, pasó de 0,571 a 0,880. Etapa B: R^2 de 0,788 a 0,791 y error típico de un factor de ×2,28 a ×2,23 sobre las vistas. Se agregaron intervalos de predicción con cobertura de 0,901 y una interfaz web consumible (sección 4).
¿Qué retos existieron en la totalidad del proyecto?	Dieciséis retos documentados en cinco frentes: calidad de los datos, fuga de información, desbalance y calidad de las etiquetas, techo de los modelos clásicos, incertidumbre de la predicción, infraestructura de cómputo, despliegue y reproducibilidad (sección 5).

Pregunta	Respuesta
¿Cómo se solucionaron?	Con decisiones verificables: registro de limpieza con respaldo de los registros retirados, exclusión de variables con fuga, validación cruzada con las mismas particiones en todo el proyecto, validación anidada, ajuste fino de un transformer en GPU, *stacking*, predicción conformal, MLflow y un notebook único en Colab (Tabla 5).
¿Cuánto cambió la solución final respecto al diseño inicial?	Se conservaron la arquitectura en dos etapas, el objetivo, la métrica principal, el protocolo de validación y el conjunto de variables. Cambiaron los dos modelos, se agregaron la cuantificación de la incertidumbre, la trazabilidad de los datos y el despliegue (sección 6, Tabla 6).
¿Qué lecciones aprendidas quedaron?	Diez lecciones sobre datos, modelado, evaluación y ingeniería; la principal es que el modelo estaba limitado por la información disponible y no por su configuración (sección 7).
Trabajo colaborativo	El proyecto se desarrolló de forma individual con aprobación del docente. Se aplicaron las prácticas que permiten la colaboración: control de versiones en GitHub, código modular, notebook reproducible en Colab, bitácora de experimentos y documentación (sección 8).
Repositorio y Colab	https://github.com/Nicolas2322/proyecto-estadisticas-youtube; notebook final `notebooks/Avances_IV_proyecto_final.ipynb`, con botón para abrirlo en Google Colab (sección 9).

3. Diseño Inicial y Solución Final

El diseño inicial (Avances II) planteó una arquitectura en dos etapas que responde a la naturaleza del problema: el sentimiento se observa en cada comentario, mientras que el desempeño se mide por video. La etapa A clasifica el sentimiento de cada comentario; sus predicciones se agregan por video junto con variables de interacción (likes de los comentarios y su concentración), polaridad léxica, tópicos latentes del texto, antigüedad y categoría; y la etapa B estima con ellas el logaritmo de las vistas. La solución final conserva esta arquitectura y modifica sus componentes, como se resume a continuación.



Se mantuvo deliberadamente la exclusión de los likes y del número de comentarios del video como predictores. Con ellos el R^2 sube a 0,944, pero esa cifra no corresponde a una predicción útil: los likes y las vistas son casi la misma señal (correlación de Spearman de 0,95) y ambos se conocen al mismo tiempo, de modo que un modelo que los use no anticipa nada. La Tabla 2 detalla el diseño inicial y la solución final por componente.

Tabla 2

Componentes del diseño inicial y de la solución final

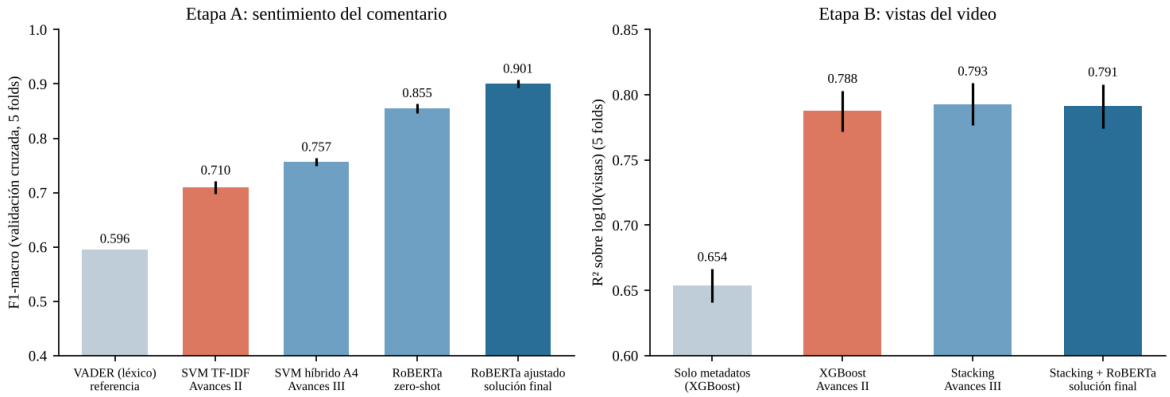
Componente	Diseño inicial (Avances II)	Solución final (Avances IV)
Preparación de datos	Limpieza descrita en el informe: duplicados, contadores ocultos, valores faltantes.	Registro automático por paso con conteos antes y después, criterio aplicado, distinción entre duplicados exactos y lógicos, huellas SHA-256 y CSV de respaldo para revertir, según la fase de preparación de CRISP-DM (Chapman et al., 2000).
Etapa A: modelo	SVM lineal con TF-IDF de palabras y caracteres, pesos de clase balanceados.	RoBERTa-base (Liu et al., 2019) preentrenado con 124 millones de tuits (Loureiro et al., 2022), ajustado 2 épocas con pérdida ponderada por clase.
Etapa B: modelo	XGBoost con hiperparámetros ajustados por búsqueda en malla.	*Stacking* de cuatro modelos con meta-modelo RidgeCV (Wolpert, 1992).
Variables de la etapa B	32 variables (31 numéricas y la categoría).	Las mismas 32 variables; se probaron y descartaron título, *embeddings* y selección por SHAP.
Incertidumbre	No contemplada.	Intervalo conformal del 90 % por validación cruzada (Angelopoulos y Bates, 2021).
Validación	Validación cruzada de 5 folds y partición reservada.	Las mismas particiones (semilla 42) en todo el proyecto, validación anidada para todo componente ajustable y pruebas de significancia.
Seguimiento	Resultados en archivos JSON.	MLflow como bitácora de experimentos (Kreuzberger et al., 2023) y resultados versionados en GitHub.
Entorno	Notebook en CPU.	Notebook único en Google Colab con GPU T4, datos en Google Drive.
Consumo del modelo	No contemplado.	Interfaz web en Gradio con enlace público y registro de cada consulta.

4. Resultados frente al Diseño Inicial

Todos los resultados se obtuvieron con validación cruzada de cinco particiones y las mismas semillas desde Avances II, de modo que las cifras de las tres entregas son directamente comparables: cada mejora se mide sobre los mismos comentarios y videos de evaluación. Los resultados de la solución final provienen de la ejecución completa del notebook en Google Colab. La Figura 1 muestra la evolución del desempeño de ambas etapas a lo largo del proyecto.

Figura 1

Evolución del desempeño de la etapa A y de la etapa B a lo largo del proyecto



Análisis: en la etapa A la mejora fue escalonada: la ingeniería de variables de Avances III (preprocesamiento para redes sociales, variables léxicas, *embeddings* GloVe y ajuste anidado de umbrales) agregó 0,049 de F1-macro al diseño inicial, y el cambio de representación a un modelo de lenguaje preentrenado agregó 0,144 adicionales. Incluso sin entrenamiento sobre las etiquetas del proyecto (*zero-shot*), RoBERTa superó en 0,098 al mejor modelo clásico. En la etapa B la mejora fue pequeña pero consistente: el modelo de Avances II ya estaba cerca del límite que permite la información disponible, y la ganancia provino de combinar modelos con errores diversos y no de nuevas variables. La referencia con solo metadatos (0,654) muestra que el texto de los comentarios aporta cerca de 0,14 de R^2.

4.1 Etapa A: Clasificación de Sentimiento

La Tabla 3 compara el diseño inicial con la solución final en las métricas de la etapa A. El F1-macro, que promedia por igual las tres clases, fue la métrica principal desde el diseño inicial porque las clases están desbalanceadas (61,9 % positivo, 25,3 % neutral y 12,8 % negativo).

Tabla 3

Etapa A: resultados del diseño inicial y de la solución final (validación cruzada de 5 folds)

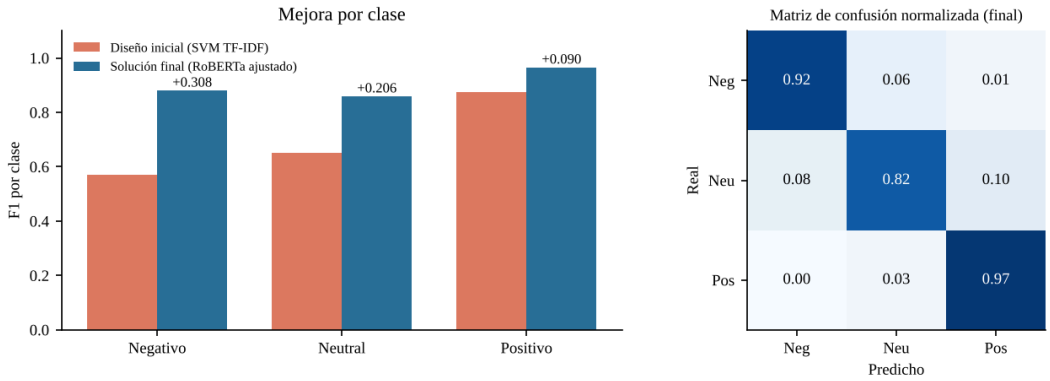
Métrica	Diseño inicial (SVM)	Avances III (mejor clásico)	Solución final (RoBERTa)	Cambio frente al inicial
F1-macro	0,710 ± 0,012	0,757 ± 0,008	0,901 ± 0,008	+0,191
Exactitud	0,784	0,820	0,927	+0,143
F1 clase negativa	0,571	0,664	0,880	+0,308
F1 clase neutral	0,653	0,706	0,858	+0,206
F1 clase positiva	0,875	0,900	0,965	+0,090

Métrica	Diseño inicial (SVM)	Avances III (mejor clásico)	Solución final (RoBERTa)	Cambio frente al inicial
F1-macro por fold		—	0,908; 0,904; 0,897; 0,907; 0,888	

Nota. El diseño inicial corresponde a la SVM lineal con TF-IDF de palabras y caracteres de Avances II. El mejor modelo clásico de Avances III (A4) combina TF-IDF, variables léxicas, la categoría del video, *embeddings* GloVe-Twitter y umbrales por clase ajustados con validación anidada.

Figura 2

F1 por clase del diseño inicial y de la solución final, y matriz de confusión de la solución final



Análisis: la mejora se concentró en la clase negativa, que en el diseño inicial era la más débil por ser la minoritaria y por depender del contexto (ironía, crítica cortés, quejas sobre el audio). La matriz de confusión muestra que los errores restantes se dan sobre todo entre clases adyacentes (neutral frente a positivo y negativo frente a neutral) y casi nunca entre negativo y positivo, el tipo de error que un usuario percibiría como grave. La dispersión entre folds es baja (desviación de 0,008), lo que indica que el resultado no depende de una partición favorable. Debe tenerse presente una cautela documentada en Avances III: el modelo preentrenado sin ajuste coincide con la etiqueta original en el 89 % de los comentarios, lo que sugiere que las etiquetas del conjunto pudieron generarse con un clasificador automático semejante; parte del desempeño puede reflejar esa afinidad y no solo la comprensión del lenguaje.

4.2 Etapa B: Desempeño del Video

La Tabla 4 compara el modelo de desempeño del diseño inicial con el de la solución final. El objetivo es $\log_{10}$(vistas), elegido en el diseño inicial porque las vistas siguen una distribución log-normal; por eso el error absoluto medio (MAE) se interpreta como un factor multiplicativo: un MAE de 0,35 equivale a que la predicción típica se desvía por un factor de $10^{0,35} \approx \times 2,2$ de las vistas reales.

Tabla 4

Etapa B: resultados del diseño inicial y de la solución final (validación cruzada de 5 folds)

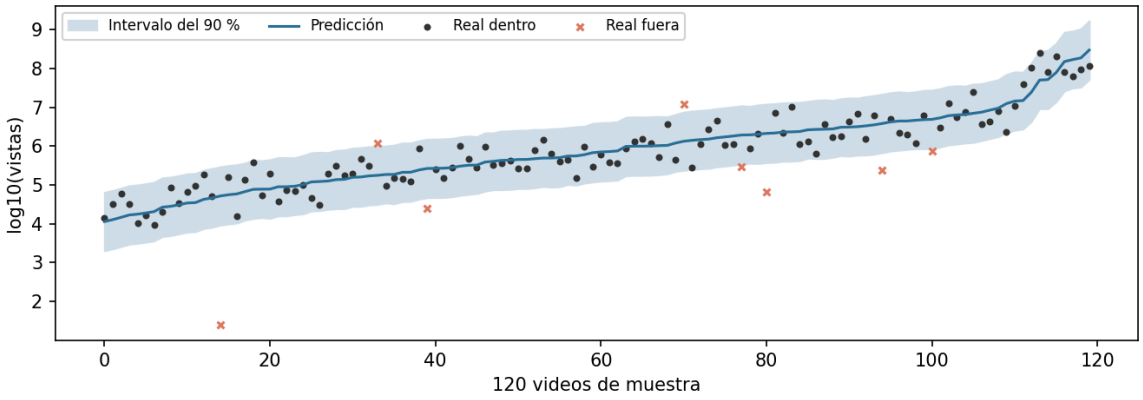
Métrica	Diseño inicial (XGBoost)	Avances III (*stacking*)	Solución final
R^2 sobre $\log_{10}$(vistas)	0,788 ± 0,016	0,793 ± 0,016	0,791 ± 0,017
MAE en $\log_{10}$ (factor sobre las vistas)	0,357 (×2,28)	0,347 (×2,22)	0,349 (×2,23)
Cobertura del intervalo del 90 %	No contemplado	0,903	0,901
Ancho medio del intervalo (factor)	No contemplado	±0,78 (×6,1)	±0,75 (×5,6)
R^2 sin variables de sentimiento (ablación)	—	—	0,793 ± 0,020
Clasificación de éxito relativo (ROC-AUC)	0,863	0,879	0,879

Nota. La solución final usa como variables de sentimiento las predicciones fuera de fold de RoBERTa ajustado. La ablación retira las siete variables de sentimiento y polaridad (proporciones de comentarios positivos y negativos, VADER y TextBlob) y conserva interacción, tópicos, antigüedad y categoría. El éxito relativo indica si el video supera la mediana de vistas de su categoría.

La ablación muestra que, al retirar las siete variables de sentimiento y polaridad, el R^2 pasa de 0,791 a 0,793 (+0,002), una diferencia menor que la variación entre folds. Del mismo modo, el R^2 de la solución final (0,791) es equivalente al del *stacking* de Avances III (0,793), que usaba las proporciones de sentimiento de la SVM: sustituir el clasificador por RoBERTa no modifica la predicción del alcance, tal como había anticipado la iteración B1 de Avances III. Este resultado confirma, con el mejor clasificador de sentimiento disponible, el hallazgo del diseño inicial: el tono de los comentarios tiene una relación débil con el alcance (correlación parcial de 0,05 con las vistas, controlando categoría y antigüedad), mientras que la interacción que generan los comentarios es la señal dominante (0,59 para el comentario más votado). Sin las variables de likes de los comentarios, el R^2 desciende a 0,674 (Avances III). Un clasificador de sentimiento mucho mejor es valioso por sí mismo, como producto de análisis de la conversación, pero no se traduce en una mejor predicción de las vistas.

Figura 3

Predicción e intervalo conformal del 90 % para 120 videos de muestra (solución final)



Análisis: los videos están ordenados por su predicción; la franja es el intervalo del 90 % y los puntos son las vistas reales. La mayoría de los valores reales cae dentro de la franja, y los que quedan fuera (marcados con una equis) se reparten a ambos lados, sin un sesgo sistemático hacia la sobre o la subestimación. El ancho del intervalo, de ±0,75 en escala logarítmica, equivale a un rango de un factor de ×5,6 hacia cada lado: el modelo distingue con claridad un video de miles de vistas de uno de millones, pero no puede precisar el valor exacto. Esta incertidumbre, ausente en el diseño inicial, se comunica al usuario en la interfaz junto con cada predicción.

4.3 Modelo Consumible

El docente indicó que el modelo debía poder consumirse por medio de una interfaz o una API. El notebook final entrena el modelo de sentimiento con todos los comentarios y el modelo de desempeño con todos los videos, y publica una interfaz en Gradio con enlace público. El usuario pega los comentarios de un video (uno por línea, con sus likes de forma opcional), elige la categoría y los días desde la publicación, y obtiene el sentimiento de cada comentario, las vistas estimadas con su intervalo del 90 % y la proporción de comentarios positivos y negativos. La interfaz usa exactamente la misma función de construcción de variables que el entrenamiento (`src/final/final.py`), lo que evita diferencias entre el modelo evaluado y el modelo en uso, y cada consulta deja un registro en formato JSON con el número de comentarios, la categoría, la predicción y la latencia, sin guardar el texto, como base para el monitoreo. La Figura 4 muestra la interfaz en funcionamiento.

Figura 4

Interfaz web del modelo final en Gradio, ejecutada desde Google Colab



Análisis: en el ejemplo se ingresan cuatro comentarios de un video de la categoría *tutorial* publicado hace 30 días. El modelo clasifica como positivos los dos agradecimientos, como negativo la queja sobre el audio y como neutral la pregunta sin carga emocional, y estima 806.905 vistas con un intervalo del 90 % entre 145.220 y 4.483.512. El rango refleja la incertidumbre medida en la validación: la interfaz no promete una cifra exacta, sino un orden de magnitud con su margen de error. Además de la interfaz, Gradio expone la misma función como una API HTTP («Usar vía API»), de modo que otro sistema puede consumir el modelo de forma programática. El enlace público que

genera Gradio es temporal (hasta una semana) y se renueva al ejecutar el notebook; un despliegue permanente, por ejemplo en Hugging Face Spaces, es el siguiente paso natural.

4.4 Cumplimiento de lo Propuesto en el Diseño

Al cierre de Avances II se propusieron acciones de mejora de los datos y del modelo, y al cierre de Avances III se comprometió la entrega de un modelo consumible. El balance es el siguiente:

- **Cumplidas:** mejorar el clasificador de sentimiento, en especial la clase negativa; cuantificar la incertidumbre de la predicción; documentar la limpieza de forma reversible; registrar los experimentos; probar los metadatos del título como variable; y exponer el modelo en una interfaz consumible.
- **Cumplidas parcialmente:** la auditoría de las etiquetas se abordó de forma indirecta, midiendo el acuerdo entre un modelo independiente y la etiqueta original (89 %), pero no se hizo un re-etiquetado manual con kappa de Cohen.
- **No cumplidas, por depender de datos nuevos:** ampliar la muestra a cientos de comentarios por video, tomar mediciones repetidas del mismo video y detectar el idioma para un modelo multilingüe. Estas acciones requieren la API de datos de YouTube y quedan como trabajo futuro; las curvas de aprendizaje de Avances II mostraron que son la vía con mayor potencial, porque el modelo de desempeño ya no mejora con más videos del mismo tipo.

5. Retos del Proyecto y su Solución

La Tabla 5 consolida los retos que se presentaron a lo largo de las cuatro etapas, cómo se identificaron, la solución aplicada y la evidencia de que funcionó. Se incluyen tanto los retos del modelado como los de datos, infraestructura y reproducibilidad, porque estos últimos consumieron una parte importante del esfuerzo.

Tabla 5

Retos identificados en el proyecto, solución aplicada y resultado

#	Reto y cómo se detectó	Solución aplicada	Resultado o evidencia
Datos			
1	Contadores con valor −1 producían −∞ al aplicar el logaritmo y rompían el entrenamiento.	Se identificó que −1 es la convención de YouTube para un contador oculto; se recategorizó como faltante y se imputa con la mediana, en lugar de eliminar los videos.	Se conservaron 8 videos sin sesgar la muestra contra los creadores que ocultan sus métricas.
2	Videos y comentarios repetidos, ninguno de los videos como copia exacta.	Se distinguieron duplicados exactos y lógicos (mismo video bajo otra búsqueda o capturado en otro momento) y se documentaron 3 videos con vistas contradictorias como limitación de la fuente.	12 videos y 249 comentarios retirados con respaldo en CSV y conteos en el registro de limpieza.

#	Reto y cómo se detectó	Solución aplicada	Resultado o evidencia
3	Necesidad de trazabilidad: el docente pidió no eliminar faltantes sin justificación y poder revertir.	Registro automático en JSON con conteos antes y después, criterio de cada paso y huellas SHA-256 de los archivos crudos.	Cualquier paso se revierte reincorporando su CSV de respaldo; la versión exacta de los datos es verificable.
4	El acceso a Kaggle estaba bloqueado desde el entorno de trabajo.	Descarga manual del conjunto y carga desde Google Drive en Colab; los CSV no se versionan en GitHub.	El notebook verifica tamaños y conteos de clases antes de entrenar.
Modelado y evaluación			
5	Fuga de información: con los likes y comentarios del video el R^2 llegaba a 0,944.	Análisis de correlación ($\rho = 0{,}95$ con las vistas) y exclusión deliberada de ambas variables, conservando la cifra solo como referencia.	El R^2 reportado ($\approx 0{,}79$) corresponde a una predicción útil y honesta.
6	Desbalance de clases: el modelo ignoraba la clase negativa (12,8 %).	F1-macro como métrica principal, pesos de clase en la SVM y en la pérdida del transformer, y umbrales por clase con validación anidada.	F1 negativo de 0,571 a 0,880.
7	Techo de los modelos clásicos: tras seis iteraciones el F1-macro se estancó en 0,757 y el *stacking* de clasificadores no mejoró.	Cambio de representación a un modelo de lenguaje preentrenado con tuits, ajustado en GPU con las mismas particiones.	F1-macro 0,901; superior al mejor clásico en los cinco folds.
8	Posible sesgo de las etiquetas: el modelo sin ajuste coincide en un 89 % con ellas.	Se reporta explícitamente como cautela y se propone la auditoría manual como trabajo futuro.	Interpretación prudente del resultado de la etapa A.
9	Las variables nuevas (título, *embeddings*, selección por SHAP) y la optimización bayesiana no mejoraron la etapa B.	Se aceptó el diagnóstico de un modelo limitado por la información y se combinaron modelos con errores diversos mediante *stacking*.	+0,006 de R^2 y reducción significativa del error (Wilcoxon, $p \approx 2 \cdot 10^{-9}$).
10	Riesgo de sobreestimar el desempeño al ajustar hiperparámetros y umbrales sobre los mismos datos de evaluación.	Validación anidada para todo componente ajustable y mismas particiones (semilla 42) en las cuatro entregas.	Las mejoras se comparan sobre los mismos datos de prueba y con pruebas pareadas (McNemar, Wilcoxon).
11	La predicción puntual ocultaba una incertidumbre grande (factor de ×2,2 en el error típico).	Predicción conformal por validación cruzada: el ancho de cada fold se calibra con los residuos de los demás.	Cobertura empírica de 0,901 frente al 90 % nominal.
Infraestructura, despliegue y reproducibilidad			
12	El ajuste fino de un transformer no es viable en CPU.	Notebook en Google Colab con GPU T4, precisión mixta y lotes de 32.	Cinco folds y el modelo final se entrenan en una sola ejecución del notebook.
13	Diferencias entre el código de entrenamiento y el de uso del modelo.	Un único módulo (`src/final/final.py`) construye las variables tanto en la evaluación como en la interfaz.	La predicción de la demo usa exactamente las mismas transformaciones evaluadas.

#	Reto y cómo se detectó	Solución aplicada	Resultado o evidencia
14	La validación con intervalos tardaba casi 7 minutos por entrenar dos veces cada fold.	Rediseño a predicción conformal por validación cruzada con un solo ajuste por fold, reutilizando las predicciones fuera de fold.	Tiempo reducido a la mitad con cobertura equivalente (0,901 frente a 0,903).
15	Errores de versiones de librerías (codificación dispersa, parámetros de matplotlib, almacenamiento de MLflow) y caché que ignoraba cambios en la malla de búsqueda.	Corrección del código a las API actuales, MLflow con base de datos SQLite y la malla como argumento explícito de la caché.	Ejecución sin errores en Colab con las versiones vigentes.
16	Ejecuciones repetidas duplicaban registros en MLflow y se perdía la trazabilidad de qué corrida era la válida.	Respaldo de la base de experimentos antes de cada corrida y restauración ante duplicados.	Bitácora con una corrida por iteración, exportada a CSV en el repositorio.

6. Cambio de la Solución Final respecto al Diseño Inicial

Para responder cuánto cambió la solución, se clasificaron sus elementos en tres grupos: lo que se conservó, lo que se reemplazó y lo que se agregó. La Tabla 6 resume la clasificación.

Tabla 6

Magnitud del cambio entre el diseño inicial y la solución final

Tipo de cambio	Elementos	Justificación
Se conservó	Arquitectura en dos etapas; objetivo $\log_{10}$(vistas); F1-macro y R^2 como métricas principales; protocolo de validación cruzada de 5 folds con las mismas semillas; las 32 variables de la etapa B; exclusión de las variables con fuga.	Las iteraciones de Avances III no encontraron alternativas mejores: el diseño era correcto.
Se reemplazó	Clasificador de sentimiento (SVM → RoBERTa ajustado); modelo de desempeño (XGBoost → *stacking* de cuatro modelos); entorno (CPU local → Google Colab con GPU).	Mejoras medidas sobre los mismos datos de prueba y estadísticamente significativas.
Se agregó	Registro de limpieza reversible; intervalos conformales; bitácora de experimentos en MLflow; pruebas de significancia; interfaz consumible con registro de consultas; módulo único de construcción de variables.	Requisitos que el diseño inicial no contemplaba: trazabilidad, incertidumbre y despliegue.
Se descartó	Variables del título, *embeddings* GloVe en la etapa B, selección de variables por SHAP, optimización bayesiana, *stacking* de clasificadores de sentimiento y vistas por día como objetivo.	No mejoraron el desempeño en validación cruzada (Avances III).

En síntesis, la solución final cambió poco en su concepción y mucho en su ejecución. El planteamiento del problema, la arquitectura y las variables del diseño inicial resistieron 25 iteraciones, lo que valida el análisis exploratorio en que se apoyaron. Los dos modelos sí cambiaron, con un efecto muy distinto en cada etapa: en el sentimiento el cambio fue grande (+0,191 de F1-macro, una reducción de 66 % de los errores de clasificación), mientras que en el desempeño fue marginal (+0,004 de R^2), porque allí el límite lo pone la información y no el algoritmo. El mayor cambio cualitativo no está en la exactitud sino en lo que la solución entrega: una predicción con su

margen de error, unos datos auditables y un modelo que cualquier persona puede usar desde un navegador.

7. Lecciones Aprendidas

Las lecciones se agrupan según la fase del proyecto en que surgieron y se formulan de modo que sean aplicables a proyectos futuros de PLN y aprendizaje automático.

- **Entender los datos antes de modelar.** El análisis exploratorio de Avances II, en particular la correlación parcial entre el sentimiento y las vistas, anticipó correctamente que un mejor clasificador de sentimiento no mejoraría la predicción del alcance. Esa predicción se confirmó en Avances III y en la solución final.
- **Un valor faltante también es información.** El −1 de YouTube significa «contador oculto», no cero ni error; tratarlo como faltante evitó eliminar videos y sesgar la muestra.
- **Desconfiar de los resultados demasiado buenos.** Un R^2 de 0,944 era fuga de información. Antes de celebrar un resultado hay que preguntarse si la variable estaría disponible en el momento de predecir.
- **Las mismas particiones para todo.** Usar las mismas semillas y la validación anidada desde el inicio hizo que cada mejora fuera comparable y que las pruebas pareadas fueran válidas, lo que evitó discusiones sobre si una diferencia era real.
- **Diagnosticar el límite antes de optimizar.** Ni la búsqueda en malla ni la optimización bayesiana mejoraron la etapa B: el modelo estaba limitado por la información. Las curvas de aprendizaje y la ablación indican dónde invertir el esfuerzo.
- **La representación importa más que el clasificador.** Seis iteraciones de ingeniería de variables aportaron 0,049 de F1-macro; cambiar a una representación preentrenada aportó 0,144. En PLN, empezar por un modelo preentrenado como línea base es una decisión que ahorra tiempo.
- **Combinar modelos solo sirve cuando sus errores son diversos.** El *stacking* mejoró la etapa B, donde los modelos de árboles y el lineal se equivocan en videos distintos, pero no la etapa A, donde todos los clasificadores clásicos cometían los mismos errores.
- **Una predicción sin incertidumbre es incompleta.** Un error típico de ×2,2 cambia la forma de usar el modelo: sirve para ordenar y para distinguir órdenes de magnitud, no para prometer una cifra exacta.
- **El mismo código para entrenar y para servir.** Centralizar la construcción de variables en un módulo evitó que la demo calculara las variables de una forma distinta a la evaluada.
- **La reproducibilidad es parte del entregable.** El registro de limpieza, MLflow, el control de versiones y un notebook que corre de punta a punta en Colab permitieron rehacer cada resultado y responder a las observaciones del docente sin rehacer el trabajo.

8. Trabajo Colaborativo

La entrega admite grupos de hasta tres estudiantes. El proyecto se desarrolló de forma individual, con la aprobación previa del docente, por lo que todas las etapas (análisis, modelado, evaluación, despliegue y documentación) estuvieron a cargo del autor. Sin un equipo con el cual dividir el trabajo, se adoptaron las prácticas que hacen que un proyecto de ciencia de datos sea colaborativo, es decir, que otra persona pueda revisarlo, reproducirlo y continuarlo sin depender de su autor:

- **Control de versiones en GitHub.** Repositorio público con historial de 9 confirmaciones que documentan la evolución por entrega (datos y modelos de Avances II, iteraciones y registro de limpieza de Avances III, transformer, pipeline final y demo de Avances IV). Los datos crudos no se versionan, pero el repositorio incluye las instrucciones para obtenerlos y sus huellas SHA-256.
- **Código modular y documentado.** El código se organiza por etapa en `src/` (análisis de Avances II, iteraciones de Avances III, módulo final), con documentación en cada archivo y un README con la estructura y las instrucciones de ejecución.
- **Entorno compartible.** El notebook final abre en Google Colab con un clic desde el repositorio, instala sus dependencias, clona el código y lee los datos desde Google Drive; cualquier integrante o revisor obtiene los mismos resultados.
- **Bitácora de experimentos.** MLflow registra cada iteración con sus parámetros, métricas y decisión, y su exportación a CSV queda en el repositorio, de modo que un integrante nuevo puede ver qué se probó y por qué se descartó.
- **Resultados versionados.** Los resultados de cada entrega se guardan en JSON junto a las figuras, lo que permite comparar entregas y retomar el trabajo desde cualquier punto.
- **Retroalimentación incorporada.** Las observaciones del docente en los encuentros sincrónicos (trazabilidad de la limpieza, duplicados exactos frente a lógicos, posibilidad de revertir, modelo consumible) se incorporaron como cambios concretos y verificables en el repositorio.

Si el proyecto se ampliara a un equipo, la estructura actual permite repartir el trabajo por etapa (datos, sentimiento, desempeño y despliegue) con ramas y solicitudes de integración en GitHub, usando las mismas particiones y la bitácora de MLflow como contrato común para comparar resultados.

9. Herramientas y Reproducibilidad

La Tabla 7 resume las herramientas empleadas en la solución final y la forma de reproducirla.

Tabla 7

Herramientas, recursos del repositorio y pasos de ejecución

Recurso	Detalle
Repositorio GitHub	https://github.com/Nicolas2322/proyecto-estadisticas-youtube

Recurso	Detalle
Notebook final (Colab)	https://colab.research.google.com/github/Nicolas2322/proyecto-estadisticas-youtube/blob/main/notebooks/Avances_IV_proyecto_final.ipynb
Módulo de la solución	`src/final/final.py`: carga y limpieza, variables por comentario y por video, tópicos, *stacking*, validación con intervalos conformales y predicción para un video nuevo.
Librerías	PyTorch y Hugging Face Transformers (Wolf et al., 2020), scikit-learn (Pedregosa et al., 2011), XGBoost (Chen y Guestrin, 2016), vaderSentiment (Hutto y Gilbert, 2014), TextBlob, MLflow, Optuna, SHAP, Gradio y pandas.
Cómputo	Google Colab con GPU NVIDIA Tesla T4; tiempo de ejecución completa cercano a 35 minutos.
Pasos de ejecución	(1) Copiar `videos-stats.csv` y `comments.csv` en `MyDrive/youtube-statistics/`. (2) Abrir el notebook desde el enlace y seleccionar un entorno con GPU T4. (3) Ejecutar todo y autorizar el acceso a Google Drive. (4) Usar el enlace público que imprime la celda de Gradio.
Resultados	`reports/avances4/resultados_finales.json`, figuras y notebook ejecutado en el repositorio; copia en `MyDrive/youtube-statistics/salida_final/`.

10. Conclusiones

- La solución final cumple el objetivo del proyecto: estima las vistas de un video a partir de sus comentarios con un R^2 de 0,791 sobre el logaritmo de las vistas, un error típico de un factor de ×2,23 y un intervalo del 90 % con cobertura empírica de 0,901, y se puede consumir desde una interfaz web.
- Frente al diseño inicial, el clasificador de sentimiento mejoró de forma sustancial (F1-macro de 0,710 a 0,901, F1 negativo de 0,571 a 0,880) gracias al cambio a un modelo de lenguaje preentrenado; el modelo de desempeño mejoró de forma marginal (R^2 de 0,788 a 0,791) porque está limitado por la información disponible.
- El planteamiento del diseño inicial se conservó: la arquitectura en dos etapas, el objetivo, las métricas, el protocolo de validación y las variables resistieron todas las iteraciones. Los cambios se concentraron en los modelos y en lo que se agregó: trazabilidad de los datos, cuantificación de la incertidumbre, bitácora de experimentos y despliegue.
- El sentimiento de los comentarios tiene poco poder para predecir el alcance de un video; la interacción que generan los comentarios, la antigüedad y la categoría concentran la señal. Mejorar la predicción requiere datos nuevos (más comentarios por video y mediciones en el tiempo) y no algoritmos más complejos.
- Los retos más costosos no fueron algorítmicos sino de datos, de evaluación honesta y de reproducibilidad, y su solución dejó prácticas que se pueden aplicar a cualquier proyecto: registro reversible de la limpieza, mismas particiones para todo, validación anidada, detección de fuga y un único código para entrenar y servir.

11. Referencias

Angelopoulos, A. N. y Bates, S. (2021). *A gentle introduction to conformal prediction and distribution-free uncertainty quantification*. arXiv. https://doi.org/10.48550/arXiv.2107.07511

Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C. y Wirth, R. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS.

Chen, T. y Guestrin, C. (2016). XGBoost: A scalable tree boosting system. En *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). https://doi.org/10.1145/2939672.2939785

Hutto, C. J. y Gilbert, E. (2014). VADER: A parsimonious rule-based model for sentiment analysis of social media text. En *Proceedings of the Eighth International AAAI Conference on Weblogs and Social Media* (pp. 216–225).

Kreuzberger, D., Kühl, N. y Hirschl, S. (2023). Machine learning operations (MLOps): Overview, definition, and architecture. *IEEE Access, 11*, 31866–31879. https://doi.org/10.1109/ACCESS.2023.3262138

Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L. y Stoyanov, V. (2019). *RoBERTa: A robustly optimized BERT pretraining approach*. arXiv. https://doi.org/10.48550/arXiv.1907.11692

Loureiro, D., Barbieri, F., Neves, L., Espinosa Anke, L. y Camacho-Collados, J. (2022). TimeLMs: Diachronic language models from Twitter. En *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics: System Demonstrations* (pp. 251–260). https://doi.org/10.18653/v1/2022.acl-demo.25

Patil, A. (2022). *YouTube statistics* [Conjunto de datos]. Kaggle. https://www.kaggle.com/datasets/advaypatil/youtube-statistics

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. y Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., . . . Rush, A. M. (2020). Transformers: State-of-the-art natural language processing. En *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations* (pp. 38–45). https://doi.org/10.18653/v1/2020.emnlp-demos.6

Wolpert, D. H. (1992). Stacked generalization. *Neural Networks, 5*(2), 241–259. https://doi.org/10.1016/S0893-6080(05)80023-1